3.1 From fully-connected layers to convolutions

- to this day, the models that we have discussed so far remain appropriate options when we are dealing with tabular data
- by tabular, we mean that the data consist of rows corresponding to examples and columns corresponding to features
- with tabular data, we might anticipate that the patterns we seek could involve interactions among the features, but we do not assume any structure *a priori* concerning how the features interact
- sometimes, we truly lack knowledge to guide the construction of better architectures; in these cases, an MLP may be the best that we can do
- however, for high-dimensional perceptual data, such structure-less networks may not be suited

3.1 From fully-connected layers to convolutions

- for instance, suppose we want to distinguish cats from dogs
- say that we do a thorough job in data collection, collecting an annotated dataset of one-megapixel photographs
- this means that each input to the network has one million dimensions
- according to our discussions of parameterization cost of fully-connected layers, even an aggressive reduction to one thousand hidden dimensions would require a fully-connected layer characterized by $10^6 \times 10^3 = 10^9$ parameters
- learning the parameters of this network may turn out to be infeasible

3.1 From fully-connected layers to convolutions

- we might object to this argument on the basis that one megapixel resolution may not be necessary
- however, while we might be able to get away with one hundred thousand pixels, our hidden layer of size 1000 grossly underestimates the number of hidden units that it takes to learn good representations of images, so a practical system will still require billions of parameters
- moreover, learning a classifier by fitting so many parameters might require collecting an enormous dataset
- and yet, today, both humans and computers are able to distinguish cats from dogs quite well, seemingly contradicting these intuitions
- that is because images exhibit rich structure that can be exploited by humans and machine learning models alike
- convolutional neural networks (CNNs) are one creative way that machine learning has embraced for exploiting some of the known structure in natural images

3.1 From fully-connected layers to convolutions

- imagine that we want to detect an object in an image
- it seems reasonable that whatever method we use to recognize objects should not be overly concerned with the precise location of the object in the image; ideally, our system should exploit this knowledge
- we can draw some inspiration here from the children's game "Where's Waldo" (depicted in Figure 1)
- the game consists of a number of chaotic scenes with many activities

3.1 From fully-connected layers to convolutions

- Waldo shows up somewhere in each, typically in some unlikely location; the player's goal is to locate him
- despite his characteristic outfit, this can be surprisingly difficult, due to the large number of distractions
- however, *what Waldo looks like* does not depend upon *where Waldo is located*
- we could sweep the image with a Waldo detector that could assign a score to each patch, indicating the likelihood that the patch contains Waldo
- CNNs systematize this idea of *spatial invariance*, exploiting it to learn useful representations with fewer parameters

3.1 From fully-connected layers to convolutions

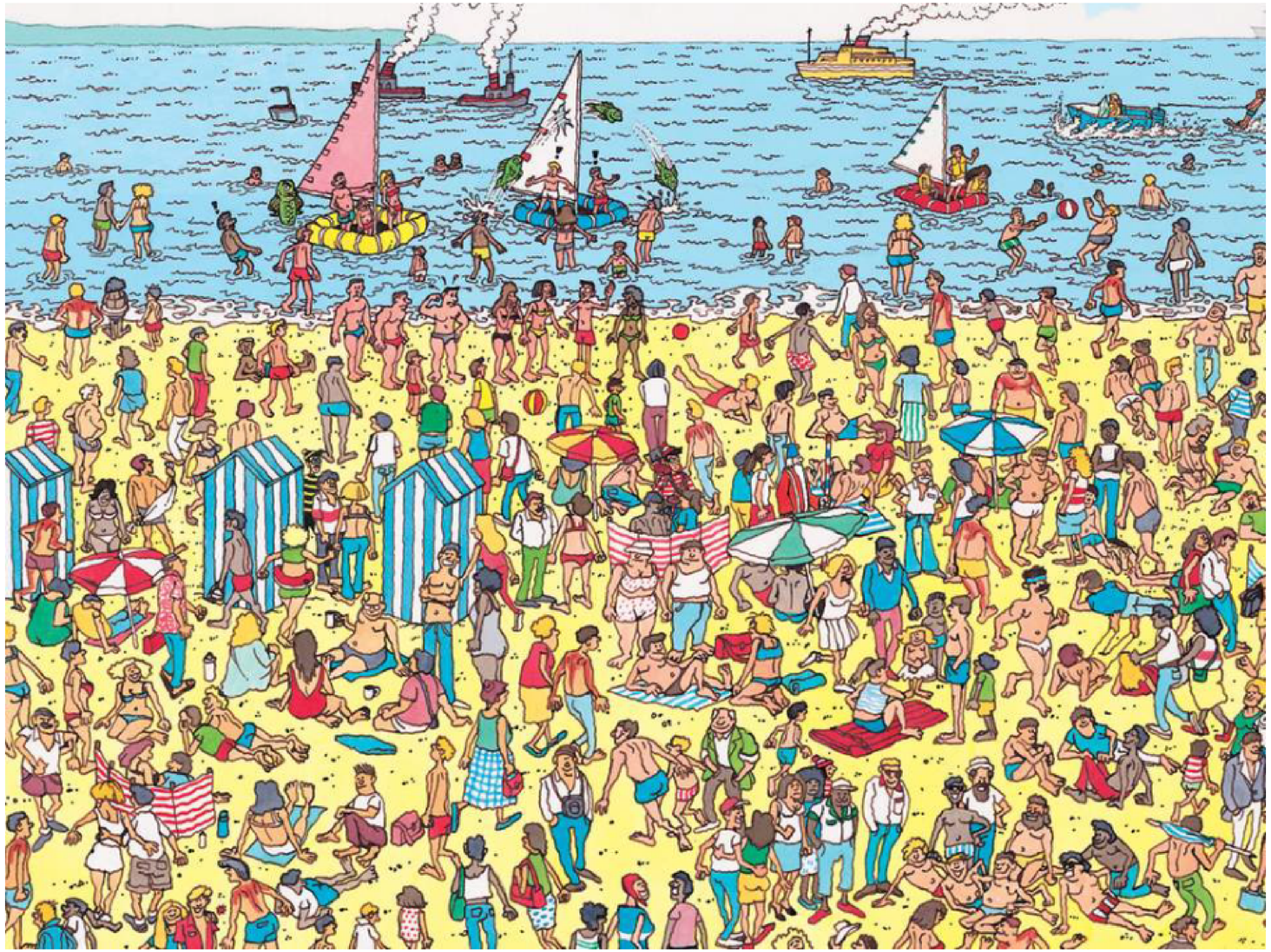

Figure 1: An image of the “Where’s Waldo” game.

3.1 From fully-connected layers to convolutions

- we can now make these intuitions more concrete by enumerating a few desiderata to guide our design of a neural network architecture suitable for computer vision:
  1. In the earliest layers, our network should respond similarly to the same patch, regardless of where it appears in the image. This principle is called *translation invariance*.
  2. The earliest layers of the network should focus on local regions, without regard for the contents of the image in distant regions. This is the *locality* principle. Eventually, these local representations can be *aggregated* to make predictions at the whole image level.
- let us see how this translates into mathematics; first, we need to discuss the notion of tensors
- just as vectors generalize scalars, and matrices generalize vectors, we can build data structures with even more axes
- *tensors* give us a generic way of describing *n*-dimensional arrays with an arbitrary number of axes
- vectors, for example, are first-order tensors, and matrices are second-order tensors
- tensors are denoted with plain capital letters (e.g., X, Y, and Z) and their indexing mechanism (e.g., x_{ijk} and $[\mathsf{X}]_{i,j,k}$) is similar to that of matrices

3.1 From fully-connected layers to convolutions

- to start off, we can consider an MLP with two-dimensional images $\mathbf{X}$ as inputs and their immediate hidden representations $\mathbf{H}$ similarly represented as matrices, where both $\mathbf{X}$ and $\mathbf{H}$ have the same shape
- we now conceive of not only the inputs, but also the hidden representations as possessing spatial structure
- let $[\mathbf{X}]_{i,j}$ and $[\mathbf{H}]_{i,j}$ denote the pixel at location (i, j) in the input image and hidden representation, respectively
- consequently, to have each of the hidden units receive input from each of the input pixels, we would switch from using weight matrices (as we did previously in MLPs) to representing our parameters as fourth-order weight tensors W

3.1 From fully-connected layers to convolutions

- suppose that **U** contains biases, we could formally express the fully-connected layer as:

$$\begin{aligned}[\mathbf{H}]_{i,j} &= [\mathbf{U}]_{i,j} + \sum_k \sum_l [\mathsf{W}]_{i,j,k,l}[\mathbf{X}]_{k,l} \\ &= [\mathbf{U}]_{i,j} + \sum_a \sum_b [\mathsf{V}]_{i,j,a,b}[\mathbf{X}]_{i+a,j+b},\end{aligned}$$

  where the switch from W to V is entirely cosmetic for now, since there is a one-to-one correspondence between coefficients in both fourth-order tensors
- we simply re-index the subscripts (k, l), such that $k = i + a$ and $l = j + b$; in other words, we set $[\mathsf{V}]_{i,j,a,b} = [\mathsf{W}]_{i,j,k,l}$
- the indices a and b run over both positive and negative offsets, covering the entire image
- for any given location (i, j) in the hidden representation $[\mathbf{H}]_{i,j}$, we compute its value by summing over pixels in **X**, centered around (i, j), and weighted by $[\mathsf{V}]_{i,j,a,b}$

3.1 From fully-connected layers to convolutions

- now, let us invoke the first principle established above: *translation invariance*
- this implies that a shift in the input **X** should simply lead to a shift in the hidden representation **H**
- this is only possible if V and **U** do not actually depend on (i, j), i.e., we have $[\mathrm{V}]_{i,j,a,b} = [\mathbf{V}]_{a,b}$ and **U** is a constant, say u
- as a result, we can simplify the definition for **H**:

$$[\mathbf{H}]_{i,j} = u + \sum_a \sum_b [\mathbf{V}]_{a,b} [\mathbf{X}]_{i+a,j+b}.$$

- this is a *convolution*: we are effectively weighting pixels at $(i + a, j + b)$, in the vicinity of location (i, j), with coefficients $[\mathbf{V}]_{a,b}$ to obtain the value $[\mathbf{H}]_{i,j}$
- note that $[\mathbf{V}]_{a,b}$ needs many fewer coefficients than $[\mathrm{V}]_{i,j,a,b}$, since it no longer depends on the location within the image, which is a significant progress

3.1 From fully-connected layers to convolutions

- now, let us invoke the second principle: *locality*
- as motivated above, we believe that we should not have to look very far away from location (i, j) in order to gather relevant information to assess what is going on at $[\mathbf{H}]_{i,j}$
- this means that, outside some range $|a| > \Delta$ or $|b| > \Delta$, we should set $[\mathbf{V}]_{a,b} = 0$
- equivalently, we can rewrite $[\mathbf{H}]_{i,j}$ as:

$$[\mathbf{H}]_{i,j} = u + \sum_{a=-\Delta}^{\Delta} \sum_{b=-\Delta}^{\Delta} [\mathbf{V}]_{a,b} [\mathbf{X}]_{i+a,j+b}. \tag{1}$$

3.1 From fully-connected layers to convolutions

- note that (1), in a nutshell, is a *convolutional layer*
- *convolutional neural networks* (*CNNs*) are a special family of neural networks that contain convolutional layers
- in the deep learning research community, **V** is referred to as a *convolution kernel*, a *filter*, or simply the layer's *weights*, which are often learnable parameters
- when the local region is small, the difference as compared with a fully-connected network can be dramatic
- while, previously, we might have required billions of parameters to represent just a single layer in an image-processing network, we now typically need just a few hundred, without altering the dimensionality of either the inputs or the hidden representations
- the price paid for this drastic reduction in parameters is that our features are now translation invariant and that our layer can only incorporate local information, when determining the value of each hidden activation

3.1 From fully-connected layers to convolutions

- before going further, we should briefly review why the above operation is called a *convolution*
- in mathematics, the convolution between two functions, say $f, g : \mathbb{R}^d \to \mathbb{R}$ is defined as:

$$(f * g)(\boldsymbol{x}) = \int f(\boldsymbol{z}) g(\boldsymbol{x} - \boldsymbol{z}) d\boldsymbol{z}.$$

- that is, we measure the *overlap* between f and g when one function is "flipped" and shifted by $\boldsymbol{x}$
- whenever we have discrete objects, the integral turns into a sum
- for instance, for vectors from the set of square summable infinite dimensional vectors with index running over $\mathbb{Z}$, we obtain the following definition:

$$(f * g)(i) = \sum_a f(a) g(i - a).$$

3.1 From fully-connected layers to convolutions

- for two-dimensional tensors, we have a corresponding sum with indices (a, b) for f and $(i - a, j - b)$ for g, respectively:

$$(f * g)(i, j) = \sum_a \sum_b f(a, b) g(i - a, j - b). \tag{2}$$

- this looks similar to (1), with one major difference
- rather than using $(i + a, j + b)$, we are using the difference instead
- note, though, that this distinction is mostly cosmetic, since we can always match the notation between (1) and (2)
- our original definition in (1) more properly describes a *cross-correlation*; we will come back to this in the following section

3.1 From fully-connected layers to convolutions

- returning to our Waldo detector, let us see what this looks like
- the convolutional layer picks windows of a given size and weighs intensities according to the filter **V**, as demonstrated in Figure 2
- we might aim to learn a model so that wherever the “Waldoness” is highest, we should find a peak in the hidden layer representations

3.1 From fully-connected layers to convolutions

Figure 2: Detecting Waldo.

3.1 From fully-connected layers to convolutions

- there is just one problem with this approach; so far, we ignored that images consist of 3 channels: red, green, and blue
- in reality, images are not two-dimensional objects but rather third-order tensors, characterized by a height, width, and channel, e.g., with shape $1024 \times 1024 \times 3$ pixels
- while the first two of these axes concern spatial relationships, the third can be regarded as assigning a multidimensional representation to each pixel location
- we thus index X as $[\mathsf{X}]_{i,j,k}$
- the convolutional filter has to adapt accordingly; instead of $[\mathbf{V}]_{a,b}$, we now have $[\mathsf{V}]_{a,b,c}$

3.1 From fully-connected layers to convolutions

- moreover, just as our input consists of a third-order tensor, it turns out to be a good idea to similarly formulate our hidden representations as third-order tensors H
- in other words, rather than just having a single hidden representation corresponding to each spatial location, we want an *entire vector* of *hidden representations* corresponding to each spatial location
- we could think of the hidden representations as comprising a number of two-dimensional grids, stacked on top of each other
- as in the inputs, these are sometimes called *channels*
- they are also sometimes called *feature maps*, as each provides a spatialized set of learned features to the subsequent layer
- intuitively, we might imagine that, at lower layers that are closer to inputs, some channels could become specialized to recognize edges, while others could recognize textures

3.1 From fully-connected layers to convolutions

- to support multiple channels in both inputs (X) and hidden representations (H), we can add a fourth coordinate to V: $[\mathrm{V}]_{a,b,c,d}$
- correspondingly, the bias u becomes a vector $\mathbf{u}$
- putting everything together, we have:

$$[\mathrm{H}]_{i,j,d} = [\mathbf{u}]_d + \sum_{a=-\Delta}^{\Delta} \sum_{b=-\Delta}^{\Delta} \sum_{c} [\mathrm{V}]_{a,b,c,d} [\mathrm{X}]_{i+a,j+b,c}, \tag{3}$$

  where d indexes the *output channels* in the hidden representations H
- the subsequent convolutional layer will go on to take a third-order tensor, H, as the input
- being more general, (3) is the definition of a *convolutional layer* for *multiple channels*, where V is a kernel or filter of the layer

3.1 From fully-connected layers to convolutions

- there are still many operations that we need to address
- for instance, we need to figure out how to combine all the hidden representations to a single output, e.g., whether there is a Waldo *anywhere* in the image
- we also need to decide how to compute things efficiently, how to combine multiple layers, appropriate activation functions, and how to make reasonable design choices to yield networks that are effective in practice
- we turn to these issues in the next sections